

Project: Designing a Telemetry and UX Database for
Chocolate-Doom Research
Part A: E-R, Data Dictionary

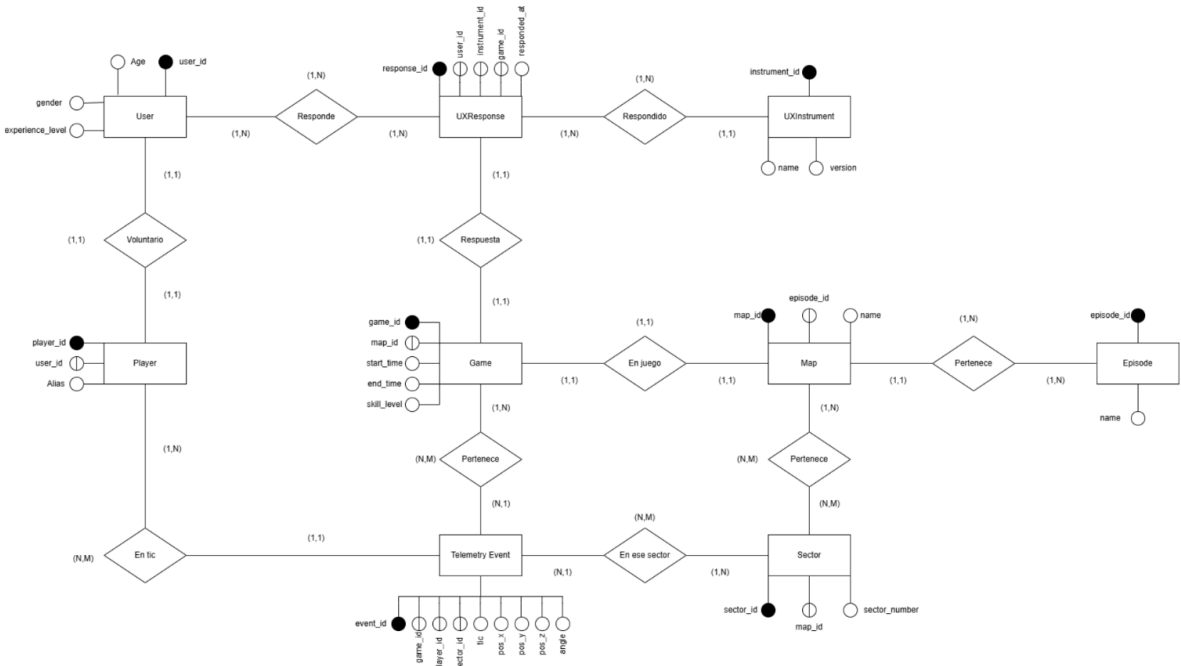
DBS – Semester Project
May 2026

Introducción

Este documento presenta el diseño de la base de datos para el análisis de telemetría y comportamiento en *Chocolate-Doom*. El objetivo es estructurar los datos capturados por *tic* y vincularlos con el perfil demográfico de los jugadores y sus resultados en las encuestas de experiencia de usuario BANGS.

A través del Diagrama Entidad-Relación, se define la jerarquía entre sesiones, mapas y registros granulares de telemetría, mientras que el Diccionario de Datos especifica las restricciones técnicas necesarias para garantizar la calidad de la información y la ética en el manejo de datos de los voluntarios.

E-R Diagrama



Diccionario

User

Tupla	Tipo	Clave	Condiciones	Descripción
user_id	Integer	PK	No Null	Identidad en el juego
age	Integer		age >= 0	Nivel de experiencia en videojuegos

gender	Varchar(20)			Género declarado
experience_level	Varchar(20)			Nivel de experiencia en videojuegos

Tupla	Tipo	Clave	Descripción
player_id	Integer	PK	Identidad en el juego
user_id	Integer	FK a user	Víncula al voluntario
alias	Varchar(50)		Nombre en el juego

Player

Episode

Tupla	Tipo	Clave	Condiciones	Descripción
episode_id	Integer	PK	No Null	Número de episodio
name	Varchar(50)			Nombre del episodio

Map

Tupla	Tipo	Clave	Condiciones	Descripción
map_id	Integer	PK	No Null	ID único del mapa
episode_id	Integer	FK a Episode	No Null	Episodio al que pertenece
name	Varchar(50)			Nombre del mapa

Sector

Tupla	Tipo	Clave	Condiciones	Descripción
sector_id	Integer	PK	No Null	ID del sector
map_id	Integer	FK a Map	No Null	Mapa al que pertenece
sector_number	Integer			Número de sector en el mapa

Game

Tupla	Tipo	Clave	Condiciones	Descripción
-------	------	-------	-------------	-------------

game_id	Integer	PK	No Null	Sesión de juego única
map_id	Integer	FK a Map	No Null	Mapa jugado
start_time	TIME		No Null	Inicio de la sesión
end_time	TIME		end_time > start_time	Fin de la sesión
skill_level	Integer			Dificultad seleccionada

Telemetry Event

Tupla	Tipo	Clave	Condiciones	Descripción
event_id	Integer	PK	No Null	ID del evento de telemetría
game_id	Integer	FK a game	No Null	Sesión a la que pertenece
player_id	Integer	FK a player	No Null	Jugador en ese tic
tic	INTEGER	No Null, CHECK > 0		Número de tic del motor
pos_x	NUMERIC(10,2)	NOT NULL		Posición X del jugador
pos_y	NUMERIC(10,2)	NOT NULL		Posición Y del jugador
pos_z	NUMERIC(10,2)	NOT NULL		Posición Z del jugador
angle	NUMERIC(6,2)	NOT NULL		Ángulo de visión (grados)

UXInstrument

Tupla	Tipo	Clave	Condiciones	Descripción
instrument_id	Integer	PK	No Null	ID del instrumento (ej: BANGS)
name	Varchar(20)			
version	Varchar(10)			

UXResponse

Tupla	Tipo	Clave	Condiciones	Descripción
response_id	Integer	PK	No Null	Respuesta de un usuario al instrumento
user_id	Integer	FK a user	No Null	Usuario que respondió
instrument_id	Integer	FK a instrument	No Null	Instrumento respondido
game_id	Integer	FK a game		Sesión después de la cual respondió
responded_at	TIME			Momento de respuesta

UXResponseItem

Tupla	Tipo	Clave	Condiciones	Descripción
response_id	Integer	FK a UXResponse	No Null	Respuesta de un usuario al instrumento
item_id	Integer	FK a UXItem	No Null	Pregunta que se responde
score	Integer	CHECK (1-5)	No Null	Instrumento respondido

UXItem

Tupla	Tipo	Clave	Condiciones	Descripción
item_id	Integer	PK	No Null	Pregunta
instrument_id	Integer	FK a UXInstrument	No Null	Pregunta que se responde
item_number	Integer			
question_text	Varchar(200)			

5. DDL / Constraints

El código completo de creación de tablas se encuentra en el archivo `sql/01_schema.sql` del repositorio. A continuación se muestra un fragmento representativo:

```
CREATE TABLE "User" (  
    user_id    INTEGER    PRIMARY KEY,  
    age        INTEGER    CHECK (age >= 0),  
    gender     VARCHAR(20),  
    experience_level VARCHAR(20)  
);
```

```
CREATE TABLE TelemetryEvent (  
    event_id INTEGER    PRIMARY KEY NOT NULL,  
    game_id  INTEGER    NOT NULL REFERENCES Game(game_id),  
    player_id INTEGER    NOT NULL REFERENCES Player(player_id),  
    tic      INTEGER    NOT NULL CHECK (tic > 0),  
    pos_x    NUMERIC(10,2) NOT NULL,  
    pos_y    NUMERIC(10,2) NOT NULL,  
    pos_z    NUMERIC(10,2) NOT NULL,  
    angle    NUMERIC(6,2) NOT NULL  
);
```

6. ETL Description

6.1 Pipeline

El pipeline de ingreso sigue tres etapas:

1. Carga raw: Los archivos TSV generados por el motor se cargan a una tabla staging_telemetry donde todos los campos son TEXT.
2. Validacion: Se identifican y registran en error_log_telemetry las filas con valores nulos, no numericos o fuera de rango.
3. Transformacion: Las filas validas se castean a sus tipos correctos, se duplican por (game_id, tic, player_id) y se insertan en TelemetryEvent.

6.2 Muestra de Telemetria Real

A continuacion se muestra una muestra de los datos capturados jugando Chocolate-Doom:

timestamp	tic	x	y	z	angle	momx	momy
2026-05-20 15:11:51	188	0	- 160	0	90.00	-1	1
2026-05-20 15:11:52	223	- 126	-31	0	80.16	-7	-3
2026-05-20 15:11:53	258	-77	-46	0	97.03	9	-1
2026-05-20 15:11:54	293	47	- 175	0	101.25	-1	2
2026-05-20 15:11:55	328	3	106	0	104.06	1	-7

Las sesiones reales capturadas se encuentran en la carpeta data/ del repositorio (sesion1.tsv, sesion2.tsv, sesion3.tsv, sesion4.tsv).

```
- Doom Telemetry\q
      SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public';
  table_name
-----
User
player
episode
map
uxinstrument
uxitem
sector
game
telemetryevent
uxresponse
uxresponseitem
11 rows)

Doom_telemetry=# SELECT * FROM player;
 player_id | user_id |  alias
-----+-----+-----
          1 |         1 | DoomGuy01
          2 |         2 | Slayer02
          3 |         3 | Rip03
          4 |         4 | Tear04
          5 |         5 | Frag05
          6 |         6 | Raze06
6 rows)
```



```

doom_telemetry=# SELECT 'Episode'          AS tabla, COUNT(*) AS filas FROM Episode
UNION ALL
SELECT 'Map',          COUNT(*) FROM Map
UNION ALL
SELECT 'Sector',       COUNT(*) FROM Sector
UNION ALL
SELECT '"User"',       COUNT(*) FROM "User"
UNION ALL
SELECT 'Player',       COUNT(*) FROM Player
UNION ALL
SELECT 'Game',         COUNT(*) FROM Game
UNION ALL
SELECT 'TelemetryEvent',COUNT(*) FROM TelemetryEvent
UNION ALL
SELECT 'UXInstrument',  COUNT(*) FROM UXInstrument
UNION ALL
SELECT 'UXItem',        COUNT(*) FROM UXItem
UNION ALL
SELECT 'UXResponse',    COUNT(*) FROM UXResponse
UNION ALL
SELECT 'UXResponseItem',COUNT(*) FROM UXResponseItem;
      tabla      | filas
-----+-----
Episode          |      3
Map               |      3
Sector           |      9
"User"           |      6
Player           |      6
Game             |      6
TelemetryEvent   |  20034
UXInstrument      |       1
UXItem           |       0
UXResponse        |       0
UXResponseItem   |       0
(11 rows)

doom_telemetry=#

```

Conclusión

El modelo diseñado permite integrar con éxito el flujo masivo de datos técnicos de juego con las métricas psicométricas de los voluntarios. La estructura garantiza que cada movimiento en los ejes esté correctamente asociado a un contexto de mapa y sesión específico, facilitando la detección de patrones de cooperación o sabotaje. Con este esquema, la base de datos está lista para soportar la fase de analítica y consultas de tendencias de movimiento sin comprometer la integridad ni la privacidad de los participantes.